Assignment 1

- 姓名：戴益阳
- 专业：计算机科学与技术
- 学号：24300240114

- 实验时间：2026/9/29～2026/10/12
- 指导老师：邱锡鹏
- 本次实验，我完成了所有内容。

目录

- Assignment 1
  - 目录
  - Part 1
    - 一、问答
      - 1. 指标定义
      - 2. 分类指标计算
    - 二、代码实现
  - Part 2
    - 一、分裂准则实现
    - 二、测试与日志
    - 三、四种准则结果对比与分析
  - Part 3
    - 一、代码实现
    - 二、测试与日志
    - 三、实验结果与分析
  - 实验心得

Part 1

一、问答

1. 指标定义

用自己的话简要说明各指标的含义及使用场景：

指标	含义	使用场景
Accuracy	准确率，即所有样本中预测正确的样本所占的比例：`(TP+TN) / (TP+TN+FP+FN)`。数值越高说明整体判断越准。	适用于各类别样本数量大致均衡的一般分类任务，如手写数字识别、图像分类、垃圾邮件判定等。
MSE	均方误差，预测值与真实值之差的平方的平均值：`(1/n) Σ(y_true - y_pred)²`。数值越小说明预测越接近真实值，为 0 时完全一致。	用于回归任务中衡量连续数值预测的误差，如房价预测、气温预测、销量预测等。

指标	含义	使用场景
Precision	精确率，被预测为正类的样本中真正为正类的比例：`TP / (TP+FP)`。回答的是"预测为正的到底有多少是可靠的"。	适用于误报（FP）代价较高的场景，如垃圾邮件过滤（宁可漏掉也尽量不误删正常邮件）、商品推荐、搜索引擎中不希望出现太多无关结果时。
Recall	召回率，真实为正类的样本中被正确预测为正类的比例：`TP / (TP+FN)`。回答的是"真正的正类被找出了多少"。	适用于漏报（FN）代价较高的场景，如疾病筛查、金融欺诈检测、安检等，希望尽可能把正类找出来。
F1	精确率与召回率的调和平均：`2PR / (P+R)`，综合反映精确率与召回率的平衡。F1 高说明两者都较高。	适用于类别不平衡且精确率与召回率都重要的场景，如信息检索、医疗诊断、欺诈检测等，避免只偏向其中一个指标。

2. 分类指标计算

给定数据（`1` 为正类，`0` 为负类）：

```
Y_true    = [1, 0, 1, 1, 0, 1, 0, 1, 1]
Y_predict = [1, 1, 0, 0, 0, 1, 0, 0, 0]
```

第一步：逐样本比较，判定 TP / FP / FN / TN

样本 i	1	2	3	4	5	6	7	8	9
Y_true	1	0	1	1	0	1	0	1	1
Y_predict	1	1	0	0	0	1	0	0	0
判定	TP	FP	FN	FN	TN	TP	TN	FN	FN

第二步：得到混淆矩阵各元素

- TP（真实为 1 且预测为 1）：样本 1、6，共 **2** 个；
- FP（真实为 0 但预测为 1）：样本 2，共 **1** 个；
- FN（真实为 1 但预测为 0）：样本 3、4、8、9，共 **4** 个；
- TN（真实为 0 且预测为 0）：样本 5、7，共 **2** 个。

混淆矩阵：

	预测为正（1）	预测为负（0）
真实为正（1）	TP = 2	FN = 4
真实为负（0）	FP = 1	TN = 2

第三步：计算各指标（写出计算过程）

- Accuracy：

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN} = \frac{2 + 2}{2 + 1 + 4 + 2} = \frac{4}{9} \approx 0.4444$$

- Precision：

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{2}{2 + 1} = \frac{2}{3} \approx 0.6667$$

- Recall：

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{2}{2 + 4} = \frac{2}{6} = \frac{1}{3} \approx 0.3333$$

- F1：

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \times \frac{2}{3} \times \frac{1}{3}}{\frac{2}{3} + \frac{1}{3}} = \frac{\frac{4}{9}}{1} = \frac{4}{9} \approx 0.4444$$

  也可直接用 TP/FP/FN 验证：

$$F1 = \frac{2TP}{2TP + FP + FN} = \frac{2 \times 2}{2 \times 2 + 1 + 4} = \frac{4}{9} \approx 0.4444$$

最终答案汇总

指标	Accuracy	TP	FP	FN	TN	Precision	Recall	F1
结果	4/9 ≈ 0.4444	2	1	4	2	2/3 ≈ 0.6667	1/3 ≈ 0.3333	4/9 ≈ 0.4444

二、代码实现

本部分实现了以下指标函数：

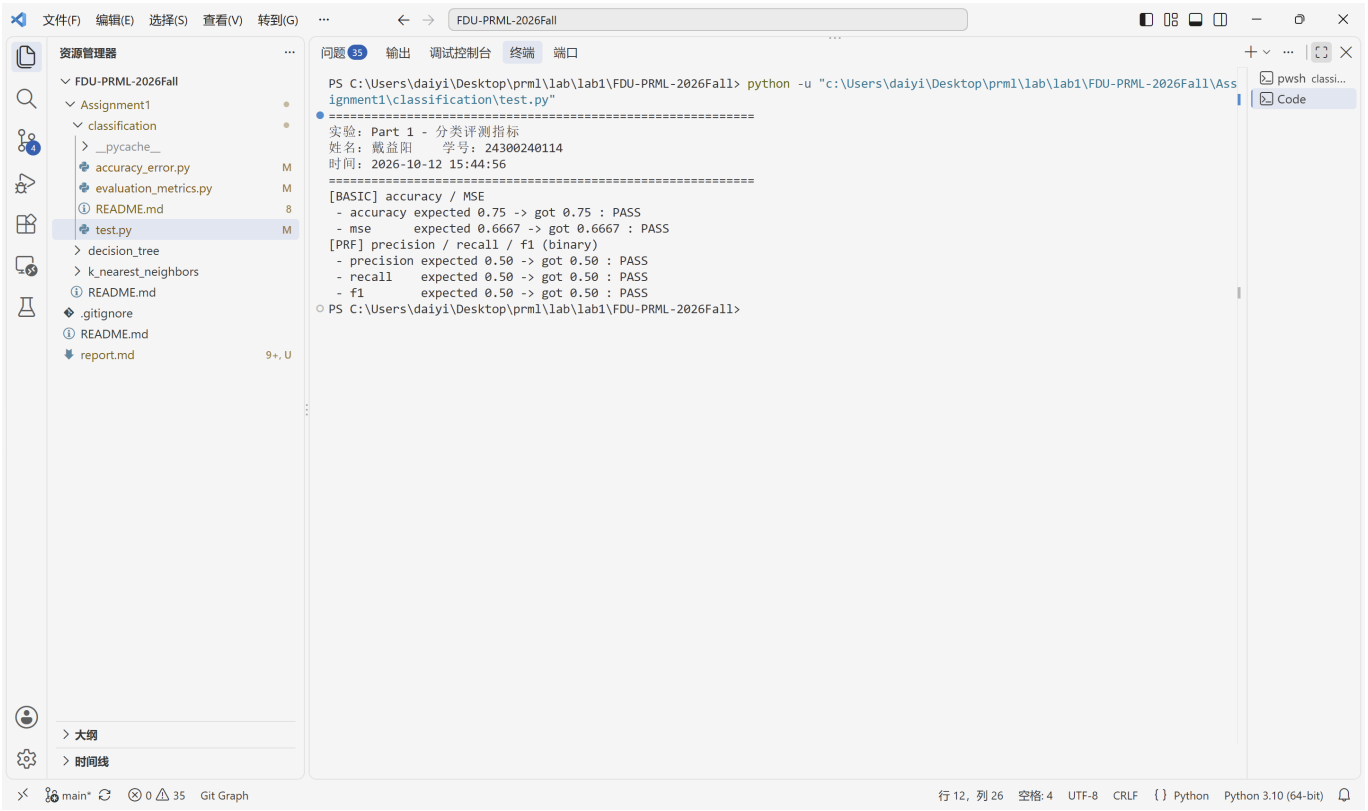
- `accuracy_error.py`：
  - `accuracy_score(y_true, y_pred)`：统计预测正确的样本数并除以样本总数，返回准确率。
  - `mean_squared_error(y_true, y_pred)`：对误差平方取平均，返回均方误差。
- `evaluation_metrics.py`：
  - `precision_score(y_true, y_pred)`：`TP / (TP + FP + EPS)`。
  - `recall_score(y_true, y_pred)`：`TP / (TP + FN + EPS)`。
  - `f1_score(y_true, y_pred)`：先算 Precision 与 Recall，再取调和平均 `2PR / (P + R + EPS)`。

其中利用已提供的 `_binary_counts` 得到 TP/FP/FN/TN，并用常量 `EPS` 避免分母为零。

测试运行结果：在 `Assignment1/classification/` 目录下运行 `python test.py`，输出如下，全部 `PASS`：

```
============================================================
实验：Part 1 - 分类评测指标
姓名：戴益阳     学号：24300240114
时间：2026-10-12 15:36:55
============================================================
[BASIC] accuracy / MSE
 - accuracy expected 0.75 -> got 0.75 : PASS
```

```
  - mse      expected 0.6667 -> got 0.6667 : PASS
[PRF] precision / recall / f1 (binary)
  - precision expected 0.50 -> got 0.50 : PASS
  - recall    expected 0.50 -> got 0.50 : PASS
  - f1        expected 0.50 -> got 0.50 : PASS
```



Part 2

一、分裂准则实现

本部分在 `criterion.py` 中实现了四种分裂准则。它们的统一思路都是：**分裂前的不纯度减去分裂后子节点按样本占比加权的平均不纯度**，得分越高表示该分裂越好。实现上新增了三个辅助函数 `_shannon_entropy`、`_gini_impurity`、`_classification_error` 分别计算单个节点的不纯度，并统一处理了"子节点为空（样本数为 0）"的边界情况。

1. Information Gain（信息增益）

信息增益定义为父节点熵减去分裂后子节点熵的加权平均：

$$\text{InfoGain} = H(D) - \sum_{v \in \{L,R\}} \frac{|D_v|}{|D|} H(D_v), \qquad H(D) = -\sum_k p_k \log_2 p_k$$

其中 p_k 为节点 D 中第 k 类样本所占比例，熵越小表示节点越纯。

2. Information Gain Ratio（信息增益率）

信息增益天然偏向取值较多的特征，因此用"分裂信息"（split information）进行归一化：

$$\text{SplitInfo} = -\frac{|D_L|}{|D|}\log_2\frac{|D_L|}{|D|} - \frac{|D_R|}{|D|}\log_2\frac{|D_R|}{|D|}, \qquad \text{GainRatio} = \frac{\text{InfoGain}}{\text{SplitInfo}}$$

当某一侧子节点为空使得 `SplitInfo = 0` 时，为避免除零错误，该分裂的增益率直接取 0。

3. Gini Index（Gini 指数）

$$\mathrm{Gini}(D) = 1 - \sum_k p_k^2, \qquad \mathrm{GiniIndex} = \mathrm{Gini}(D) - \sum_{v\in\{L,R\}} \frac{|D_v|}{|D|}\mathrm{Gini}(D_v)$$

4. Classification Error（分类误差率）

$$\mathrm{Error}(D) = 1 - \max_k p_k, \qquad \mathrm{ErrorGain} = \mathrm{Error}(D) - \sum_{v\in\{L,R\}} \frac{|D_v|}{|D|}\mathrm{Error}(D_v)$$

二、测试与日志

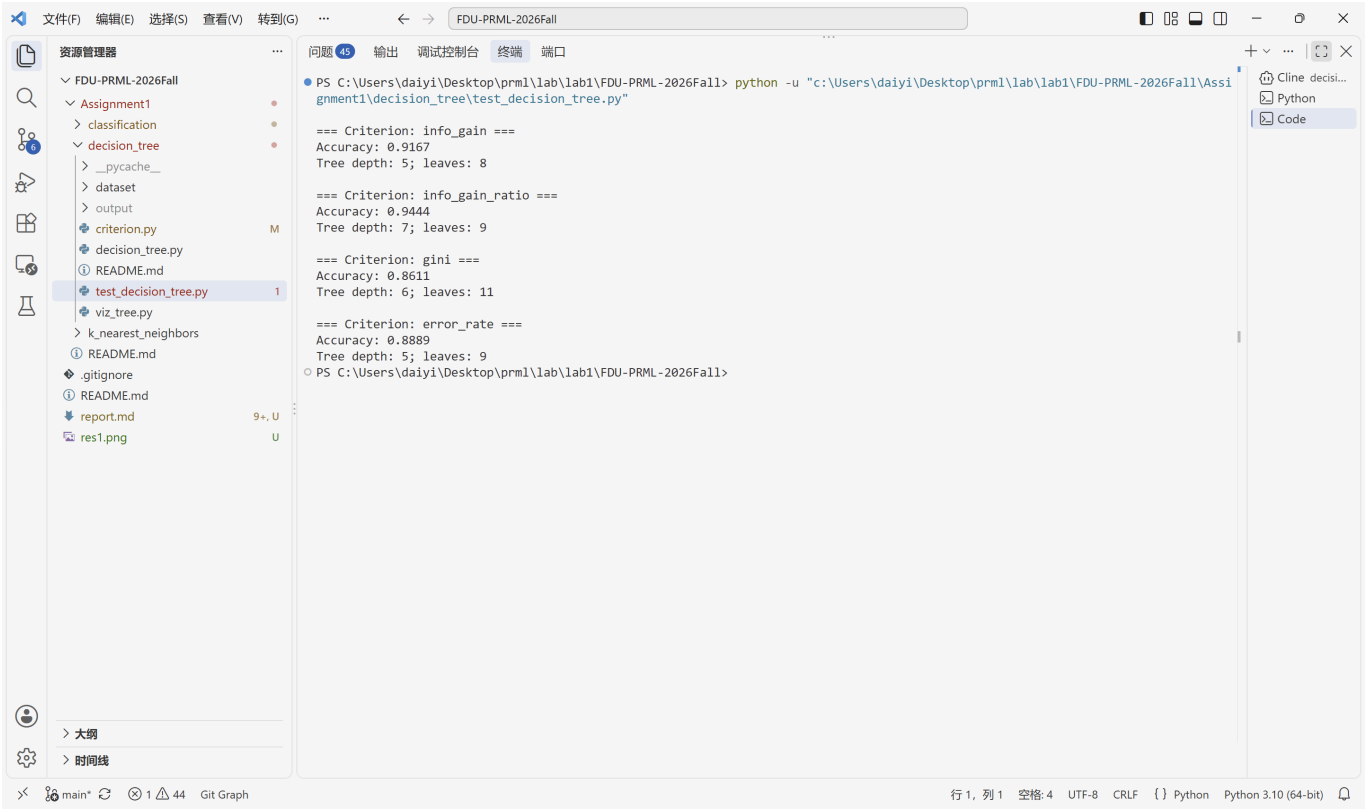
在 `Assignment1/decision_tree/` 目录下运行 `python test_decision_tree.py`，在 UCI Wine 数据集（142 条训练 / 36 条测试，3 个类别、13 个数值特征）上，四种准则均通过了测试（准确率均 ≥ 0.85），脚本同时把每棵树保存到 `output/` 目录。运行日志如下：

```
=== Criterion: info_gain ===
Accuracy: 0.9167
Tree depth: 5; leaves: 8

=== Criterion: info_gain_ratio ===
Accuracy: 0.9444
Tree depth: 7; leaves: 9

=== Criterion: gini ===
Accuracy: 0.8611
Tree depth: 6; leaves: 11

=== Criterion: error_rate ===
Accuracy: 0.8889
Tree depth: 5; leaves: 9
```



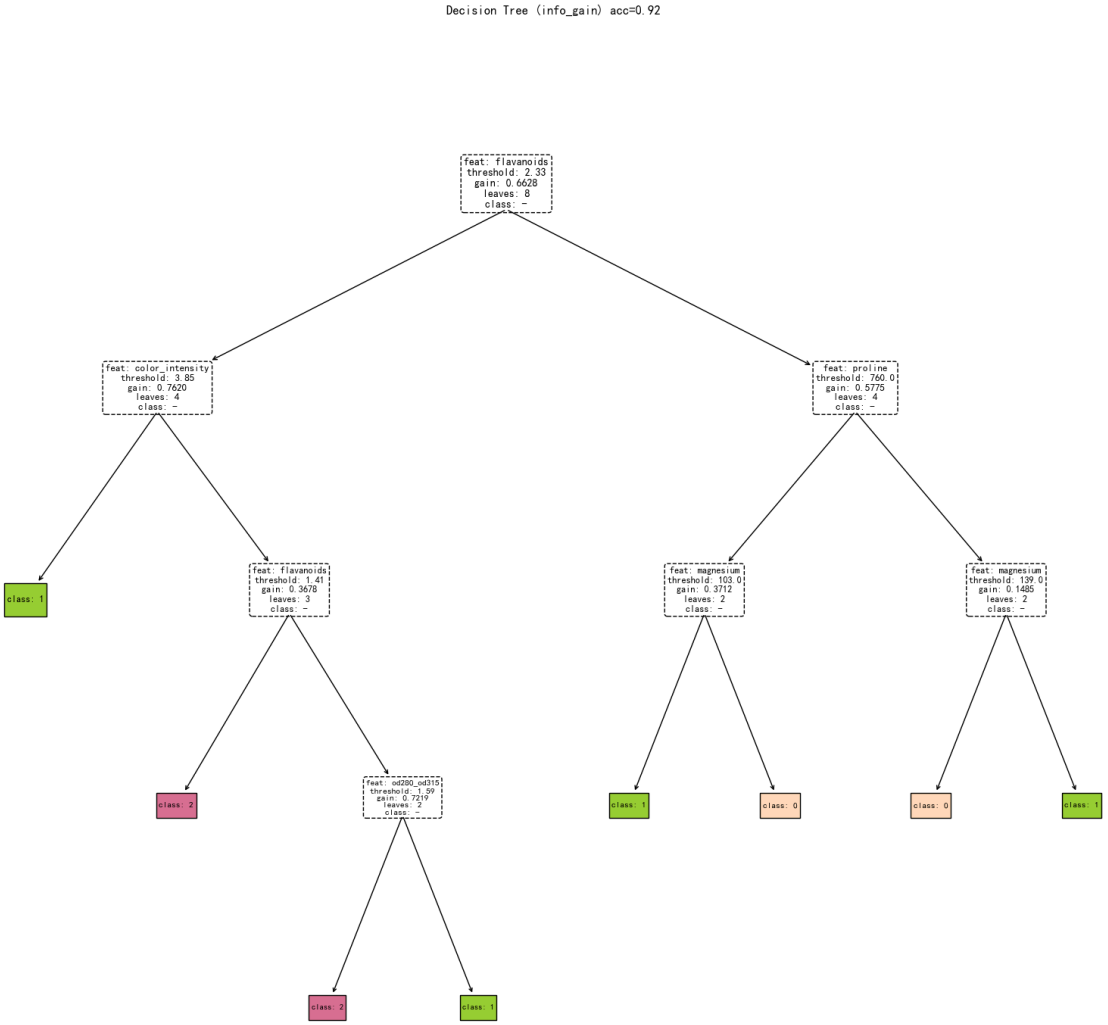
生成的决策树图像分别为：

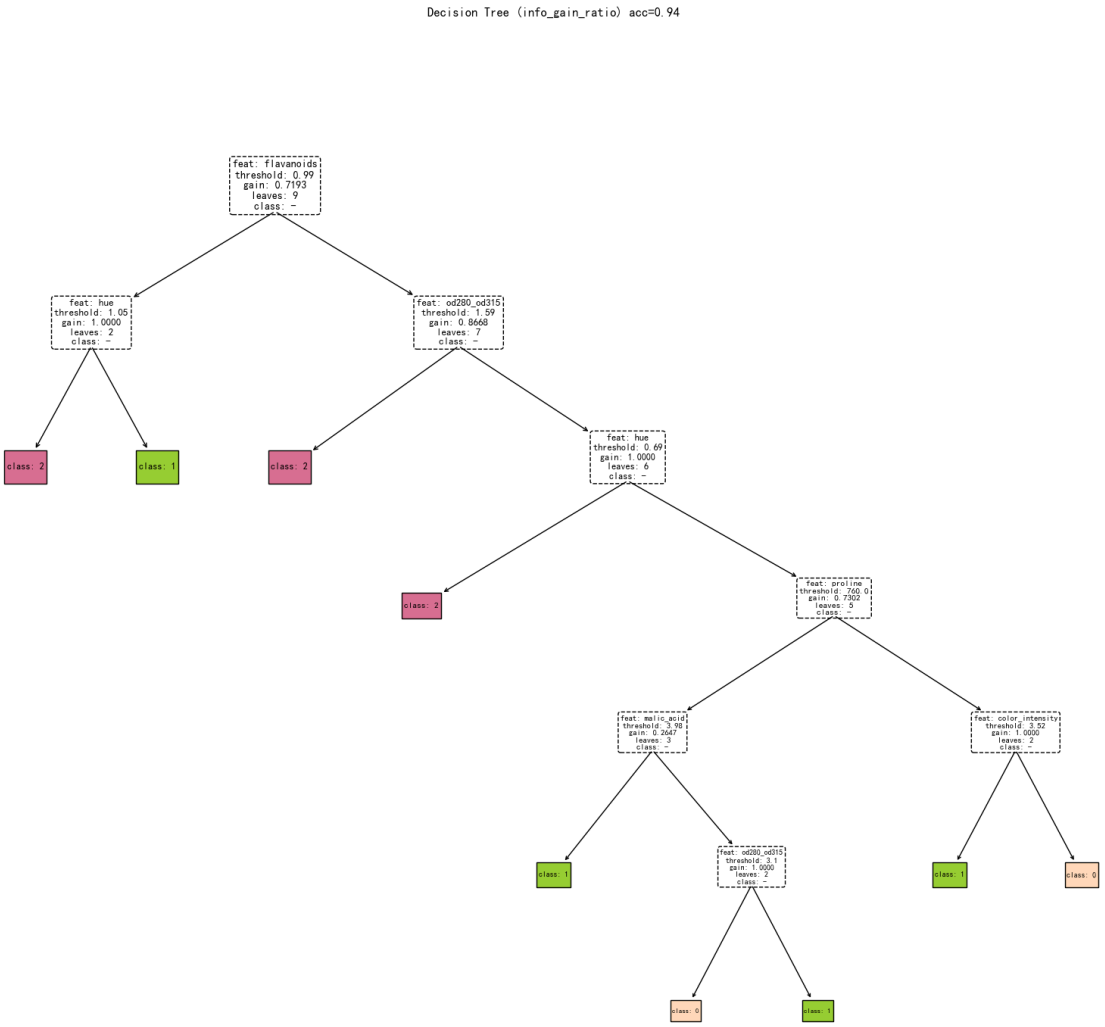
- output/wine_info_gain.png
- output/wine_info_gain_ratio.png
- output/wine_gini.png
- output/wine_error_rate.png

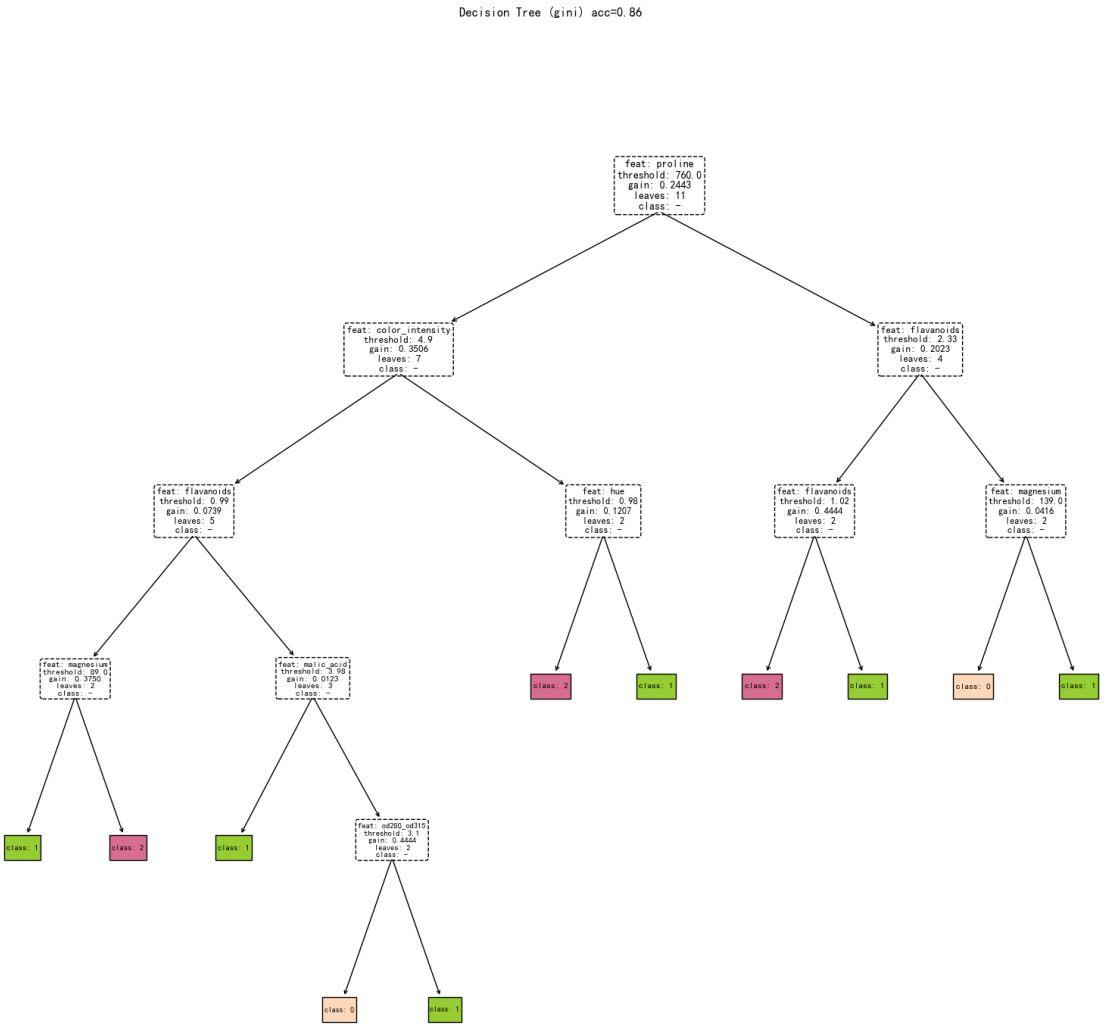
三、四种准则结果对比与分析

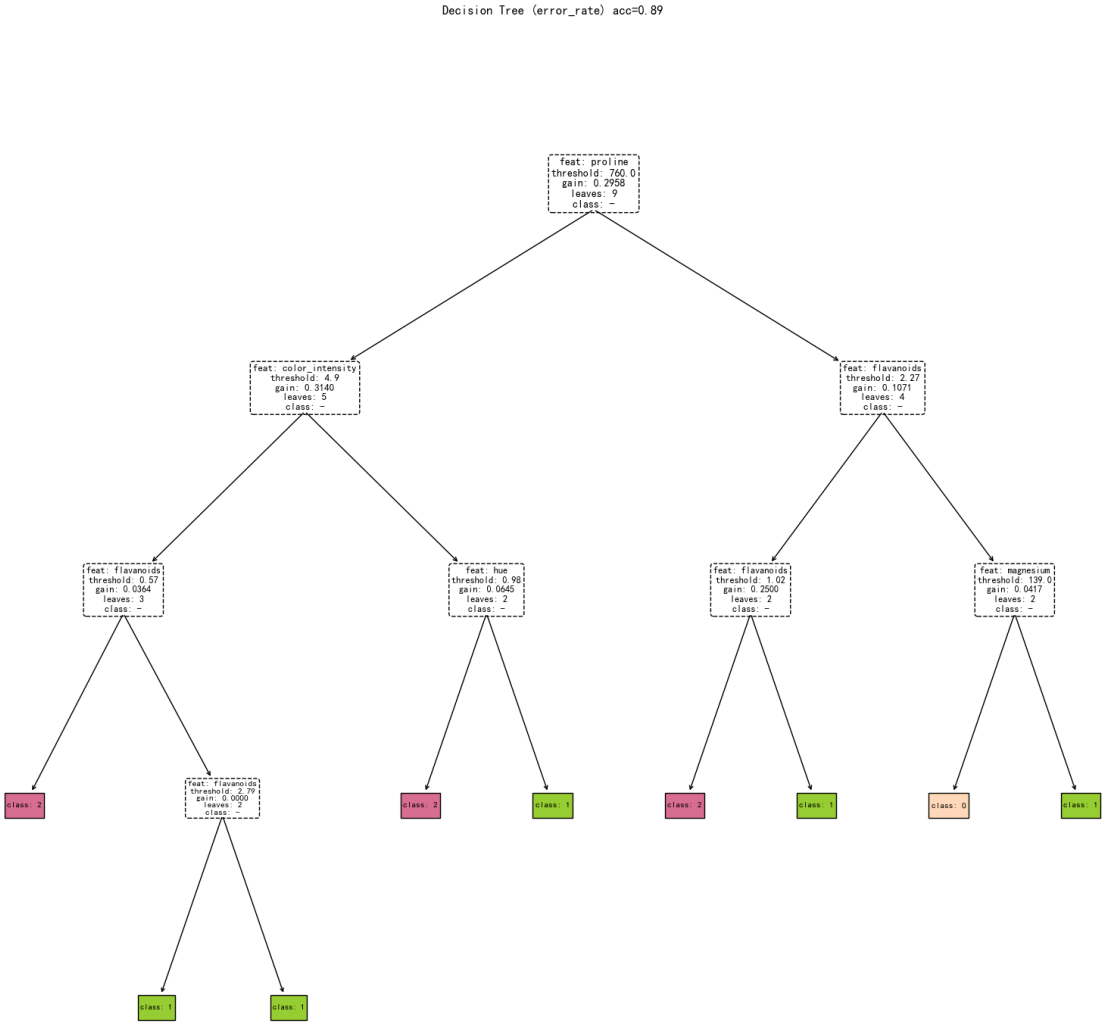
分裂准则	测试准确率	树深度	叶节点数
Information Gain	0.9167	5	8
Information Gain Ratio	0.9444	7	9
Gini Index	0.8611	6	11
Classification Error	0.8889	5	9

四种准则生成的树结构如下：


Decision Tree (info_gain) acc=0.92
feat: flavanoids
threshold: 2.33
gain: 0.6628
leaves: 8
class: –
feat: color_intensity
threshold: 3.85
gain: 0.7620
leaves: 4
class: –
feat: proline
threshold: 760.0
gain: 0.5775
leaves: 4
class: –
class: 1
feat: flavanoids
threshold: 1.41
gain: 0.3678
leaves: 3
class: –
feat: magnesium
threshold: 103.0
gain: 0.3712
leaves: 2
class: –
feat: magnesium
threshold: 139.0
gain: 0.1485
leaves: 2
class: –
class: 2
feat: od280_od315
threshold: 1.59
gain: 0.7219
leaves: 2
class: –
class: 1
class: 0
class: 0
class: 1
class: 2
class: 1


Decision Tree (info_gain_ratio) acc=0.94
feat: flavanoids
threshold: 0.99
gain: 0.7193
leaves: 9
class: -
feat: hue
threshold: 1.05
gain: 1.0000
leaves: 2
class: -
feat: od280_od315
threshold: 1.59
gain: 0.8668
leaves: 7
class: -
class: 2
class: 1
class: 2
feat: hue
threshold: 0.69
gain: 1.0000
leaves: 6
class: -
class: 2
feat: proline
threshold: 760.0
gain: 0.7302
leaves: 5
class: -
feat: malic_acid
threshold: 3.98
gain: 0.2647
leaves: 3
class: -
feat: color_intensity
threshold: 3.52
gain: 1.0000
leaves: 2
class: -
class: 1
feat: od280_od315
threshold: 3.1
gain: 1.0000
leaves: 2
class: -
class: 1
class: 0
class: 0
class: 1


Decision Tree (gini) acc=0.86
feat: proline
threshold: 760.0
gain: 0.2443
leaves: 11
class: –
feat: color_intensity
threshold: 4.9
gain: 0.3506
leaves: 7
class: –
feat: flavanoids
threshold: 2.33
gain: 0.2023
leaves: 4
class: –
feat: flavanoids
threshold: 0.99
gain: 0.0739
leaves: 5
class: –
feat: hue
threshold: 0.98
gain: 0.1207
leaves: 2
class: –
feat: flavanoids
threshold: 1.02
gain: 0.4444
leaves: 2
class: –
feat: magnesium
threshold: 139.0
gain: 0.0416
leaves: 2
class: –
feat: magnesium
threshold: 89.0
gain: 0.3750
leaves: 2
class: –
feat: malic_acid
threshold: 3.98
gain: 0.0123
leaves: 3
class: –
class: 2
class: 1
class: 2
class: 1
class: 0
class: 1
class: 1
class: 2
class: 1
feat: od280_od315
threshold: 3.1
gain: 0.4444
leaves: 2
class: –
class: 0
class: 1



分析：

1. **信息增益率效果最好（准确率 0.9444）**。它通过分裂信息对特征"取值数目偏好"做了惩罚，在 Wine 这种 13 个数值型特征的数据集上能选出更合适的分裂特征，因此准确率最高。同时这一惩罚使树分裂更"谨慎"，树更深（7 层）、叶节点数居中（9 个）。
2. **信息增益（准确率 0.9167）**：树最浅（5 层）、叶节点最少（8 个），结构最简单，也能取得不错的准确率，泛化表现稳定。
3. **Gini 指数（准确率 0.8611）**：在这组数据上准确率最低，且叶节点数最多（11 个）、树较深（6 层），树结构相对最复杂，存在一定的过拟合倾向。Gini 与熵在度量上相近，但选出的分裂点不同，对结果产生了差异。
4. **分类误差率（准确率 0.8889）**：只关注最大类占比，对概率分布的变化不如熵和 Gini 敏感，因此对候选分裂不够"挑剔"，性能居中。
5. **总体结论**：四种准则在 Wine 数据集上都能达到 0.85 以上的准确率，说明该数据集类别可分性较强。不同的不纯度度量会影响树的形状（深度、叶节点数）与分裂偏好，进而影响测试集上的泛化性能；在本实验中信息增益率综合表现最优，而 Gini 指数由于树过于复杂而略逊一筹。

Part 3

一、代码实现

本部分在 `knn_student.py` 中完成了 kNN 的"距离度量 → 近邻检索 → 投票决策"三个核心环节：

1. `pairwise_dist(X_test, X_train, metric, mode)` — 距离计算

- **L2（欧式距离）**：
  - `two_loops`：双重循环逐对计算 $\sqrt{\sum_d (x_i^{(d)} - x_j^{(d)})^2}$；
  - `no_loops`：利用展开式 $\|x_i - x_j\|^2 = \|x_i\|^2 + \|x_j\|^2 - 2\,x_i \cdot x_j$ 矩阵化一次性算出所有平方距离，由于浮点误差可能使平方距离出现接近 0 的负值，先用 `np.maximum(sq, 0)` 截断再开方。
- **cosine**：余弦距离 $= 1 - \text{cosine_similarity} = 1 - \dfrac{x_i \cdot x_j}{\|x_i\|\|x_j\|}$，分母过小时用 `eps=1e-12` 兜底防止除零。

两种 L2 实现输出完全一致（单测中 `max|diff| = 0`），且与 sklearn 参考实现逐元素误差在 `1e-13` 量级。

2. `knn_predict(X_test, X_train, y_train, k, metric, mode)` — 多数表决预测

- 对每个测试样本用 `argsort` 取距离最小的 k 个训练样本作为近邻；
- 用 `Counter` 统计 k 个近邻中各类别出现次数，得票最多的类别作为预测结果；
- 出现平票时返回**最小标签**。

3. `select_k_by_validation(X_train, y_train, X_val, y_val, ks, metric, mode)` — 验证集选参

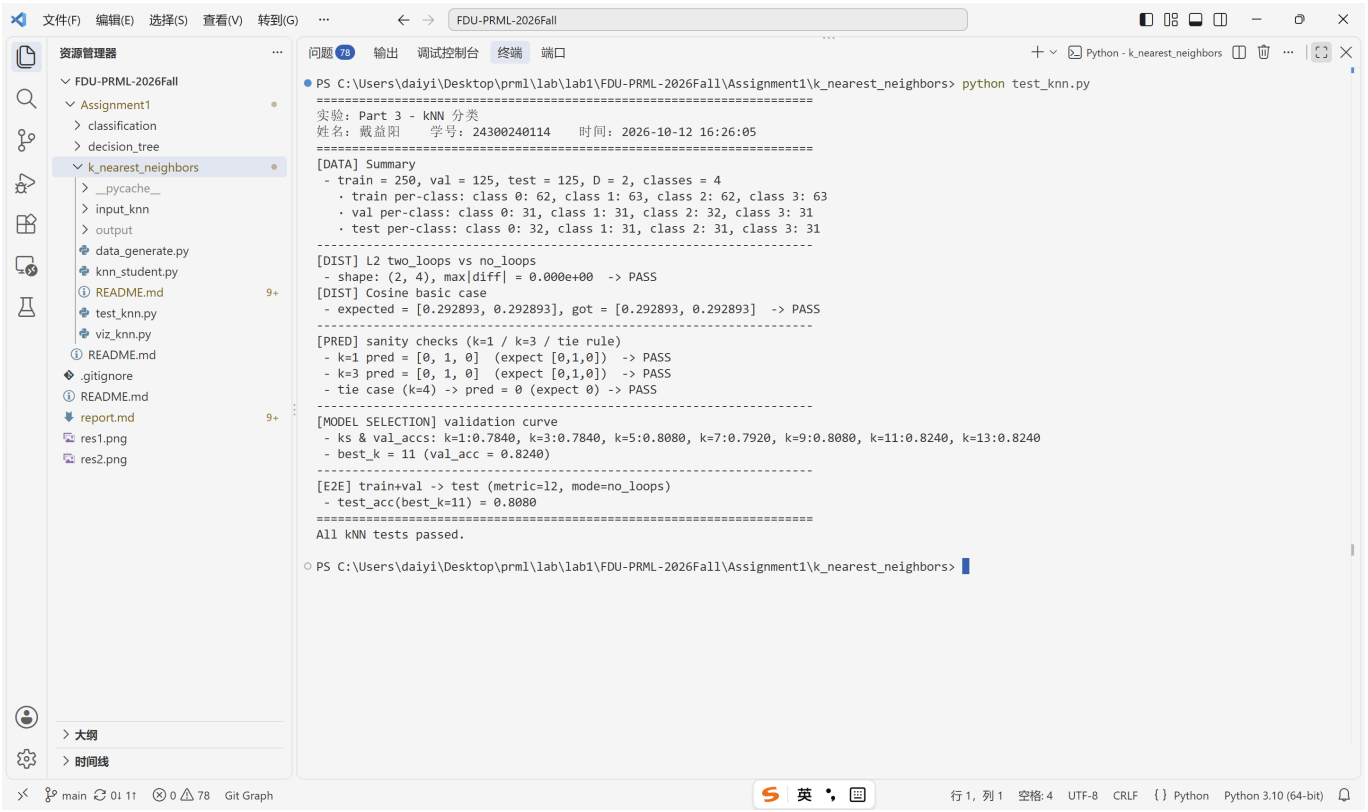
- 在 `ks` 上逐一遍历，用 `knn_predict` 计算每个 k 在验证集上的准确率，得到与 `ks` 对齐的 `accs`；
- 返回 `(best_k, accs)`；多个 k 取得相同最高验证准确率（平票）时返回**最小的 k**。

二、测试与日志

先运行 `python test_knn.py` 执行单元测试，输出如下（全部 PASS）：

```
======================================================================
实验：Part 3 - kNN 分类
姓名：戴益阳     学号：24300240114     时间：2026-10-12 16:15:27
======================================================================
[DATA] Summary
 - train = 250, val = 125, test = 125, D = 2, classes = 4
   · train per-class: class 0: 62, class 1: 63, class 2: 62, class 3: 63
   · val per-class: class 0: 31, class 1: 31, class 2: 32, class 3: 31
   · test per-class: class 0: 32, class 1: 31, class 2: 31, class 3: 31
----------------------------------------------------------------------
[DIST] L2 two_loops vs no_loops
 - shape: (2, 4), max|diff| = 0.000e+00  -> PASS
[DIST] Cosine basic case
 - expected = [0.292893, 0.292893], got = [0.292893, 0.292893]  -> PASS
----------------------------------------------------------------------
[PRED] sanity checks (k=1 / k=3 / tie rule)
 - k=1 pred = [0, 1, 0]  (expect [0,1,0])  -> PASS
 - k=3 pred = [0, 1, 0]  (expect [0,1,0])  -> PASS
 - tie case (k=4) -> pred = 0 (expect 0) -> PASS
```

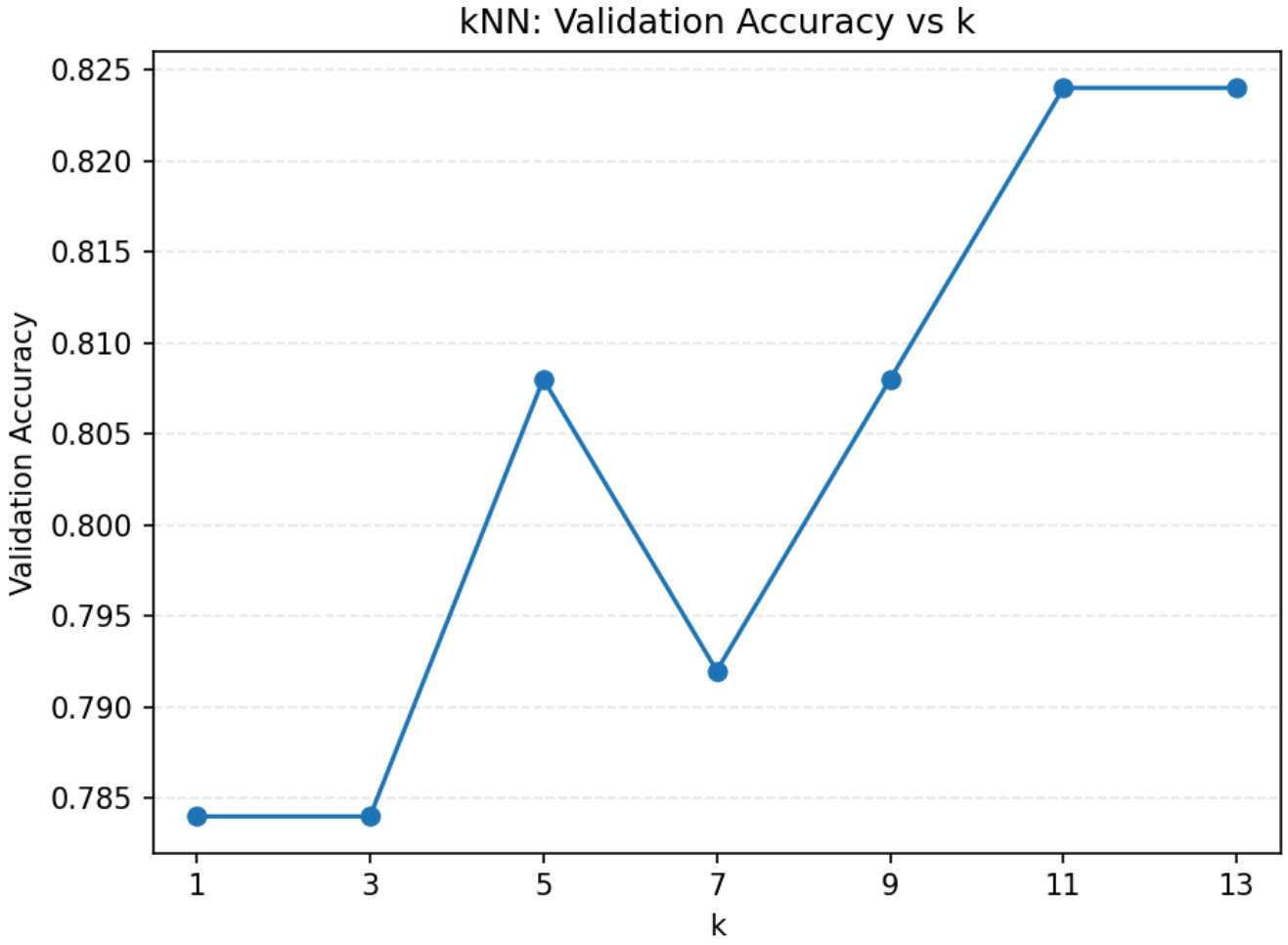
```
------------------------------------------------------------------------
[MODEL SELECTION] validation curve
 - ks & val_accs: k=1:0.7840, k=3:0.7840, k=5:0.8080, k=7:0.7920, k=9:0.8080,
k=11:0.8240, k=13:0.8240
 - best_k = 11 (val_acc = 0.8240)
------------------------------------------------------------------------
[E2E] train+val -> test (metric=l2, mode=no_loops)
 - test_acc(best_k=11) = 0.8080
========================================================================
All kNN tests passed.
```

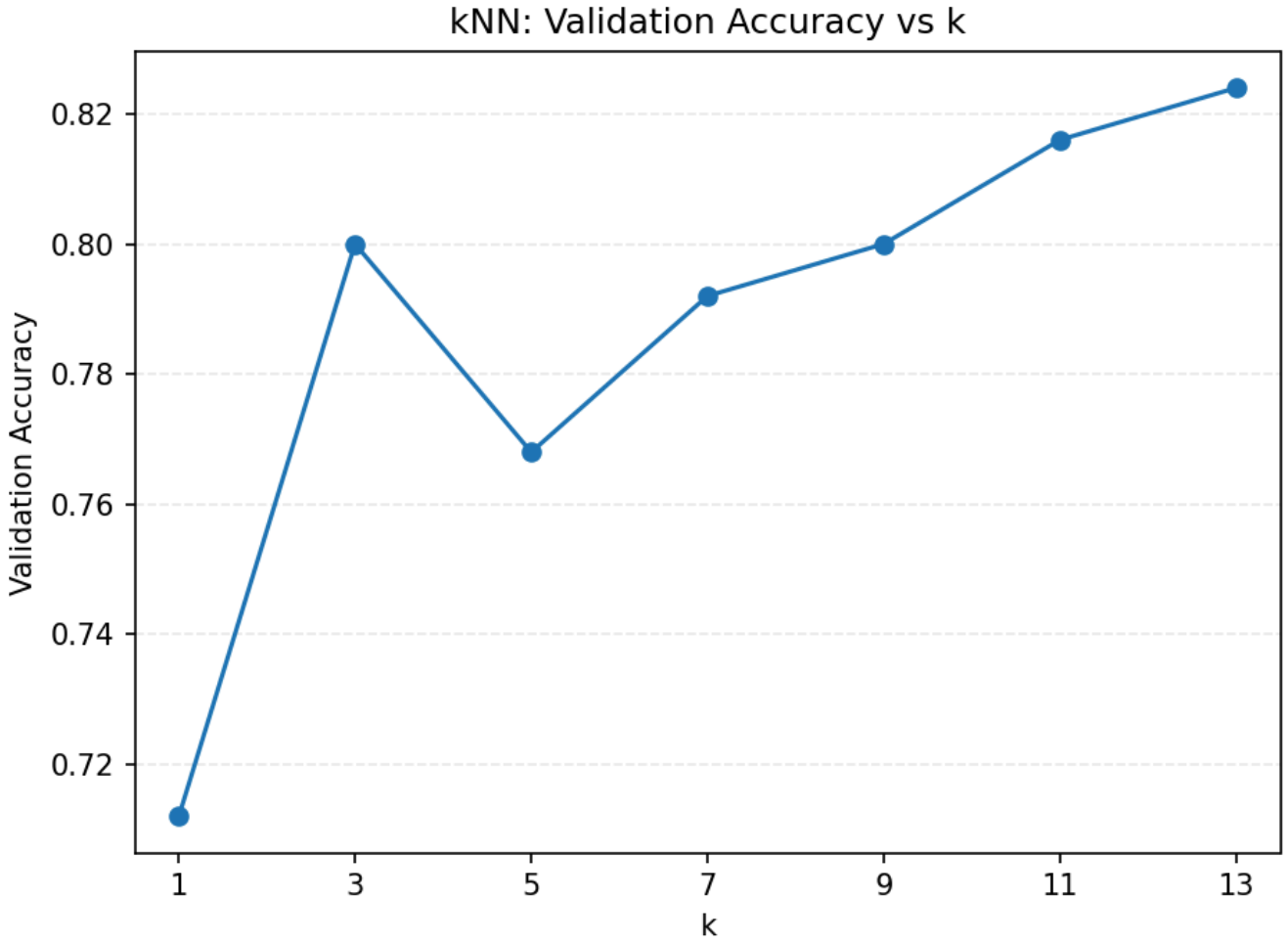


通过后运行 `python knn_student.py`，在 L2 与 cosine 两种度量下分别生成 val-accuracy vs k 曲线和并列多 K 决策边界图。

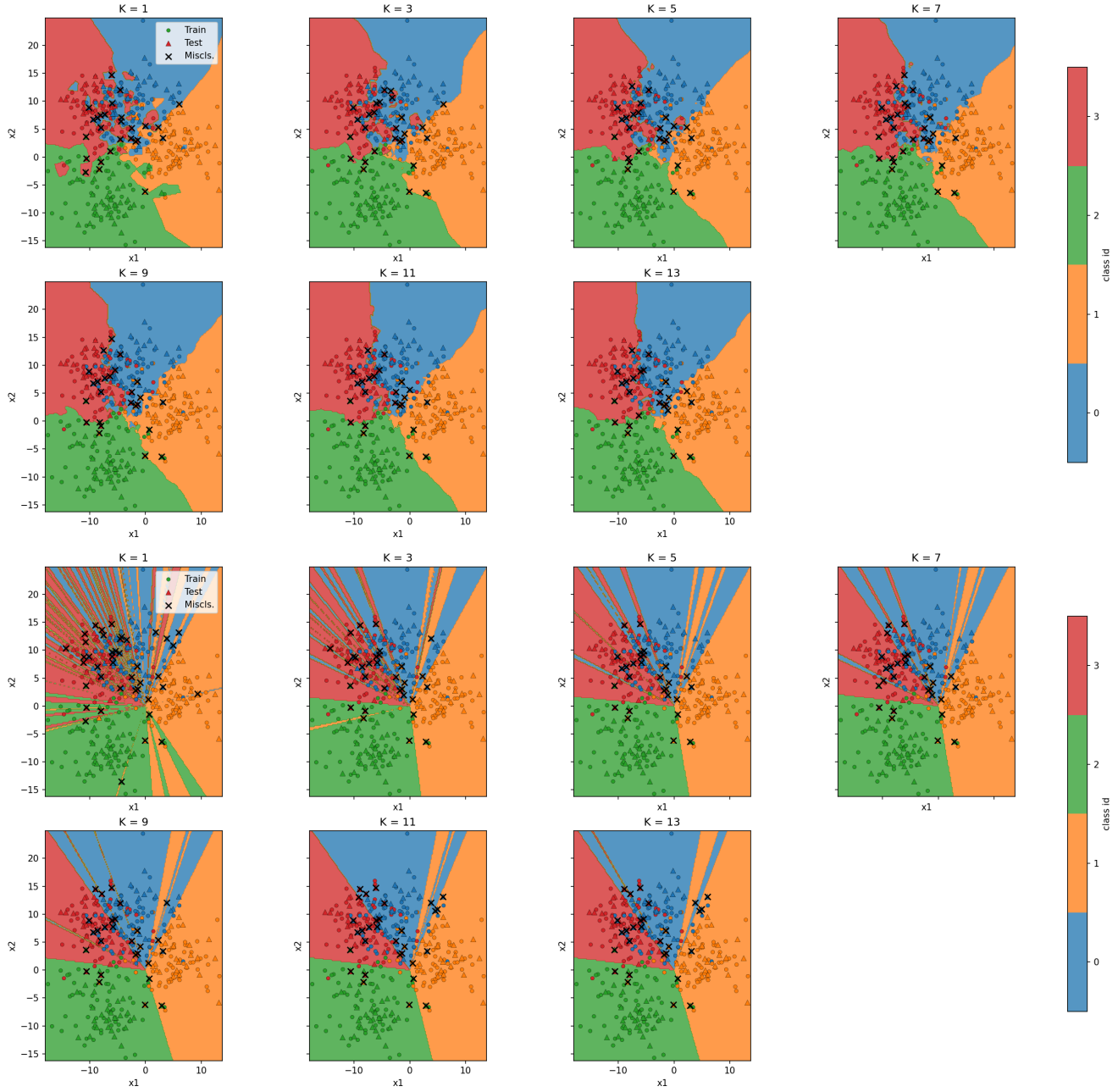
三、实验结果与分析

（一）验证准确率随 k 的变化曲线


kNN: Validation Accuracy vs k
Validation Accuracy
0.825
0.820
0.815
0.810
0.805
0.800
0.795
0.790
0.785
1
3
5
7
9
11
13
k



（二）不同 k 的决策边界（含训练/测试点与误分类点标记）



（三）数据与结果汇总

数据集由 `make_blobs` 生成：共 500 个 2D 样本、4 个类别，划分结果为训练 250 / 验证 125 / 测试 125（各类基本均衡）。两种距离度量在验证集上的网格搜索结果如下：

距离度量	验证准确率（k=1,3,5,7,9,11,13）	最优 k	验证准确率	测试准确率（train+val, best k）
L2	0.7840, 0.7840, 0.8080, 0.7920, 0.8080, **0.8240**, **0.8240**	11	0.8240	0.8080
Cosine	0.7120, 0.8000, 0.7680, 0.7920, 0.8000, 0.8160, **0.8240**	13	0.8240	0.7680

（四）分析

1. **最优 k 与测试表现**：L2 的验证集最优 k=11（0.8240），用 train+val 重新训练后在测试集上准确率为 0.8080；cosine 最优 k=13（验证 0.8240），测试准确率 0.7680。两种度量下验证准确率总体都随 k 的增

大而上升并在中等 k 处趋于稳定，说明适度增大的邻居数量有利于平滑噪声。

2. **过拟合 / 欠拟合**：从决策边界图可以看出，k=1 时边界高度曲折、几乎紧贴训练样本，对单个噪声点非常敏感，属于过拟合倾向；k 增大（如 k=7、k=11、k=13）后边界明显变平滑，泛化更好；若 k 继续无限增大，边界会过度平滑并可能丢失关键局部结构，出现欠拟合。黑色"×"标记的误分类测试点主要集中在类别交叠区域。
3. **L2 vs cosine**：该数据由球状簇生成，类别主要通过"位置（绝对坐标）"分开，欧式距离能更好刻画这种结构，因此 L2 的测试准确率（0.8080）明显高于 cosine（0.7680）；在验证集上二者最佳准确率持平（0.8240）。值得注意的是 cosine 在 k=1 时验证准确率仅 0.7120，明显低于 L2（0.7840），说明只按方向相似度选一个最近邻时更容易被干扰，需要更大的 k 才稳定。

实验心得

本次实验的三部分实验让我对有监督学习中的分类任务的两种方法：决策树和k近邻有了更深的理解。

其中，k近邻的方法早在高中信息课上就已有所介绍，非常直观容易理解，当时举的是鸢尾花分类的例子，后来在数据结构课程的期末PJ中，我又完成了高维向量检索任务的HNSW算法，知道了向量检索在高维空间中是一个很复杂很费时空的任务，因此k近邻任务并不是一个像看上去那样简单的任务。

而决策树则是我新了解的部分，实验给出了4种不同的定义划分的不纯度的方式，要求通过这些方式进行结点的划分，最终形成一棵完整的决策树，其中定义了结点的最小样本数和最大深度，但目前作用不大，也没有预剪枝和后剪枝的环节，但这个实验也让我了解了决策树的基本生成过程，也有点类似于k-d树的生成，同样是在高维空间中对各个维度进行切分。

总的来说，这次实验难度不大，但加深了我对这两种基本的分类方法的理解。